# Centralized Federated Learning Framework

**Project objective.** Develop a reusable federated learning framework in which a central coordinator trains neural-network models across multiple independently operated clients without collecting their raw data. The emphasis is on distributed systems, deployment, compatibility, observability, and robustness rather than inventing new federated learning algorithms.

**Target setting.** The project should model cross-silo federated learning: a relatively small number of organizations or sites, each controlling its own data and execution environment, participating in centrally coordinated training over real networks.

## 1. Core Requirements

- Use neural-network models. Existing ML tools such as PyTorch, scikit-learn, pandas, NumPy and established federated-learning libraries may be used.
- Support at least FedAvg and FedProx. These may be implemented directly or integrated from an existing federated-learning library.
- Implement the federation communication and orchestration layer as part of the project. Standard communication libraries and protocols, including HTTP/gRPC, serialization libraries, TLS and other secure-communication tools, may be used.
- Run the system across genuinely separate machines or virtual machines over the Internet. Suitable options include university servers, cloud instances in different regions, or team members' machines.
- Support client registration, round initiation, model/configuration distribution, local training, update collection, aggregation, evaluation, timeouts, retries, client dropout and partial participation.
- Persist enough coordinator state to resume an interrupted experiment instead of restarting training from round zero.
- Support both a local simulation mode for debugging/benchmarking and a real distributed mode. The two modes should share the same training and orchestration abstractions where practical.

## 2. Deployment and Client Compatibility

A central problem in real federated-learning deployments is ensuring that independently controlled clients execute compatible training pipelines. The framework should address this explicitly.

**Managed-code mode.** Clients use the same approved code or package. The system should provide a practical mechanism for version control and deployment, such as containers, release bundles, a private package server, or another approach that does not assume every client can access a public Git repository.

**Independent-implementation mode.** Assume that a participating organization may insist on maintaining its own implementation for security or governance reasons. Such clients should be required to satisfy a compatibility or conformance contract before joining a training run.

**The compatibility mechanism should check important assumptions such as:**

- Model architecture/version and parameter compatibility.
- Input/output shapes, feature names, ordering and data types.
- Pre-processing and categorical encoding versions.
- Feature semantics and units (for example, days versus months).
- Label semantics and expected ranges.
- Reference transformations or synthetic test cases whose expected outputs are known.

The goal is not to prescribe one exact deployment mechanism, but to ensure that incompatible clients are detected before they corrupt a federated run.

## 3. Data and Experimental Cases

The framework should be demonstrated on three different data modalities. Comparable open datasets may be substituted with approval.

| Modality | Suggested dataset | Example task | Reason for inclusion |
|---|---|---|---|
| Tabular | UCI Default of Credit Card Clients or UCI Bank Marketing | Binary classification | Banking-related data where cross-institution data sharing is a realistic concern. |
| Images | MedMNIST (for example DermaMNIST or ChestMNIST) | Medical image classification | Healthcare provides a natural multi-institution privacy setting while MedMNIST keeps compute manageable. |
| Text | UCI SMS Spam Collection | Spam classification | Telecommunications text is naturally distributed across organizations/users and raw-message sharing is sensitive. |

**Dataset references:** UCI Default of Credit Card Clients; UCI Bank Marketing; MedMNIST; UCI SMS Spam Collection

**Mandatory baseline distribution.** At minimum, clients should receive a fully randomized partition of each dataset so that the complete system can be tested in a controlled setting.

**Optional non-IID experiments.** If time permits, the team should compare the baseline against one or both of the following controlled heterogeneity cases:

- Target/label heterogeneity, including highly imbalanced clients or clients missing one or more classes.
- Feature-distribution heterogeneity, where clients observe meaningfully different input distributions.

The exact construction can be determined with supervisor input. The purpose is to evaluate system and algorithm behavior under realistic cross-silo differences, not to spend excessive effort designing synthetic partitions.

## 4. Robustness and Distributed-Systems Behavior

The system should be tested under realistic operational conditions rather than only an ideal run.

- Configurable minimum participation before a round can be aggregated.
- Slow clients and stragglers.
- Client dropout before or during a round.
- Temporary connection failures and retries.
- Duplicate or stale messages/updates where relevant to the chosen protocol.
- Client reconnection in a later round.
- Coordinator interruption and experiment resumption.

## 5. Evaluation and Metrics

The project should evaluate both machine-learning behavior and distributed-system behavior. Global performance alone is not sufficient.

### Machine-learning metrics

- Global model performance across rounds.
- Per-client model performance across rounds.
- Local training performance before/after each round where meaningful.
- Comparison of FedAvg and FedProx.
- Comparison between federated and appropriate centralized/local baselines where feasible.
- Optional comparison under the non-IID cases described above.

### Communication metrics

- Ping/round-trip measurements between coordinator and clients.
- Observed transfer latency for model/update messages.
- Message/package sizes and total bytes transmitted.
- Time required to transmit each major message type.
- Minimum, maximum, mean and other useful summary statistics.
- Client dropout, retry and failed-round statistics.

### Computation and timing metrics

- Per-round client training duration.
- Server-side aggregation/evaluation duration.
- End-to-end round duration.
- Time spent waiting for clients versus computing.
- Minimum, maximum and average timings across clients and rounds.

## 6. End-of-Training Report and Experiment Log

Every completed training session should produce a machine-readable log and a concise human-readable summary. The report should make it possible to reconstruct what happened without inspecting console output.

- Run identifier, timestamps, software/protocol/model/pre-processing versions and configuration.
- Participating clients, participation per round and dropout/failure events.
- Dataset allocation summary and sample counts per client.
- Global and per-client ML metrics over time.
- Communication metrics and their min/max/mean summaries.
- Computation and round-duration metrics and their min/max/mean summaries.
- Total number of rounds attempted/completed and reasons for failed or skipped rounds.
- Final global-model location/checksum and other artifacts required to reproduce or inspect the run.

Plots of the principal ML and systems metrics should be generated automatically at the end of training. A live dashboard is useful but optional; the end-of-training report is mandatory.

## 7. Security and Privacy Scope

The project is not intended to become a cryptography project. Standard security libraries may be used. At minimum, the students should avoid obviously unsafe transport and arbitrary unauthenticated clients; the exact authentication and secure-transport mechanism can depend on the deployment environment.

**Optional privacy extension.** Client-side clipping/noise addition or another lightweight privacy mechanism may be added and its impact on utility measured. Formal differential-privacy guarantees or privacy accounting are not required unless the team chooses this as an extension.

Secure aggregation, homomorphic encryption, Byzantine-robust aggregation, split learning and advanced privacy mechanisms are outside the core scope and may be treated as optional extensions.

## 8. Deliverables

- A reusable federated-learning framework with coordinator and client software.
- Local simulation mode and real distributed deployment mode.
- At least one real multi-machine/VM Internet deployment.
- FedAvg and FedProx support.
- Managed-code deployment/versioning mechanism and a client compatibility/conformance mechanism.
- Demonstrations on tabular, image and text datasets.
- Automated experiment logging, end-of-training report and plots.
- Tests demonstrating dropout/failure handling and interrupted-run recovery.
- Technical report describing architecture, experiments, limitations and design decisions.

## 9. Expected Outcome

The result should be a small but reusable cross-silo federated-learning platform that can coordinate multiple independently operated sites, enforce compatible training pipelines, survive realistic network/client problems, and provide enough observability to understand both learning performance and systems performance. The students are expected to provide a GitHub repository complete with instructions on how to run it.